

Secure Software Design and Development



Threat Model Report

Submitted to:

Engr Muhammad Ahmad Nawaz

Prepared by:

Maheen Aamir (SP23-BCT-027)

Malik Muhammad Huzaifa (SP23-BCT-029)

Muzamil Ali Akhtar (SP23-BCT-043)

Maryum Masood (SP23-BCT-031)

Department of computer science

(Cyber Security)

Threat Model Report

1. Scope and System Context

This threat model is based on the current source code of the Flask To-Do application. The analysis covers all authentication and task-management routes, data models, application factory, and configuration module.

1.1 Files in Scope

This threat model is based on the current code in:

- `app/routes/auth.py` — `/register`, `/login`, `/logout`
- `app/routes/tasks.py` — `/`, `/tasks/create`, `/tasks/<int:task_id>/edit`, `/tasks/<int:task_id>/delete`
- `app/models/user.py` and `app/models/task.py`
- `app/__init__.py` (application factory) and `config.py`

1.2 Data Storage

Data is persisted in SQLite (`todo.db`). The `Task.user_id` foreign key serves as the primary ownership boundary between user records. All object-level access decisions are based on this boundary.

2. STRIDE Threat Analysis

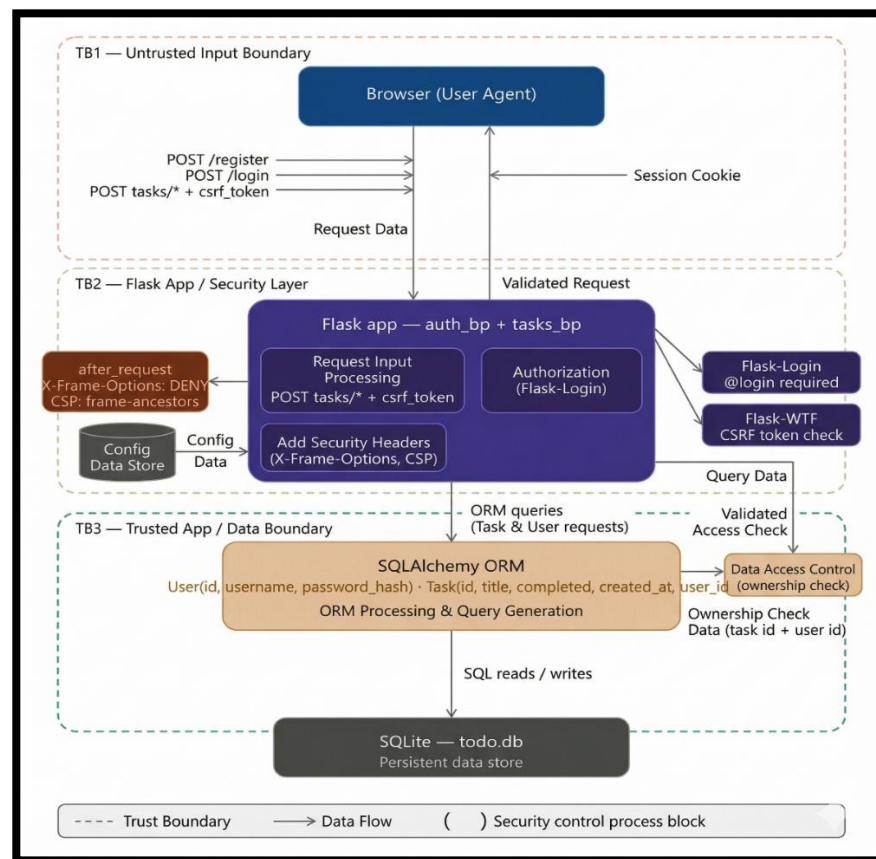
The following table presents the full STRIDE threat register for the Flask To-Do application, covering each threat category with a concrete application example and current mitigation status.

STRIDE Category	Threat Description	Example in this To-Do App	Mitigation Status in Code
S (Spoofing)	Attacker acts as another user by stealing credentials/session.	POST to <code>/login</code> with stolen credentials then access <code>/</code> to read tasks.	Passwords hashed with <code>generate_password_hash()</code> and verified with <code>check_password_hash()</code> . Authenticated routes use <code>@login_required</code> . Gap: no lockout/rate limit on login.
T (Tampering)	User alters task data they do not own.	Change <code>task_id</code> in <code>/tasks//edit</code> or <code>/tasks//delete</code> .	Mitigated: <code>_get_user_task_or_404()</code> uses <code>Task.query.filter_by(id=task_id, user_id=current_user.id).first_or_404()</code> .
R (Repudiation)	User denies making a change; no forensic trail.	User claims they did not delete a task.	Gap: only flash messages; no persistent audit log (user id, IP, timestamp, route).

STRIDE Category	Threat Description	Example in this To-Do App	Mitigation Status in Code
I (Information Disclosure)	Unauthorized data exposure between users or via unsafe runtime config.	View tasks by ID enumeration; stack traces if debug=True.	Per-user queries exist, object-level checks in place. Gap: run.py currently starts Flask with debug=True.
D (Denial of Service)	Resource exhaustion blocks legitimate use.	Repeated POST flooding on /login or saturates app/DB.	Gap: no request throttling, CAPTCHA, or reverse-proxy limits in code.
E (Elevation of Privilege)	Regular user performs actions outside their scope.	Normal user attempts to access other users' tasks.	Mitigated: task ownership enforced. No admin role currently.

3. Data Flow Diagram (DFD)

The following DFD describes the data flows between the browser, the Flask application, the ORM layer, and the SQLite database, including the security controls applied at each layer.

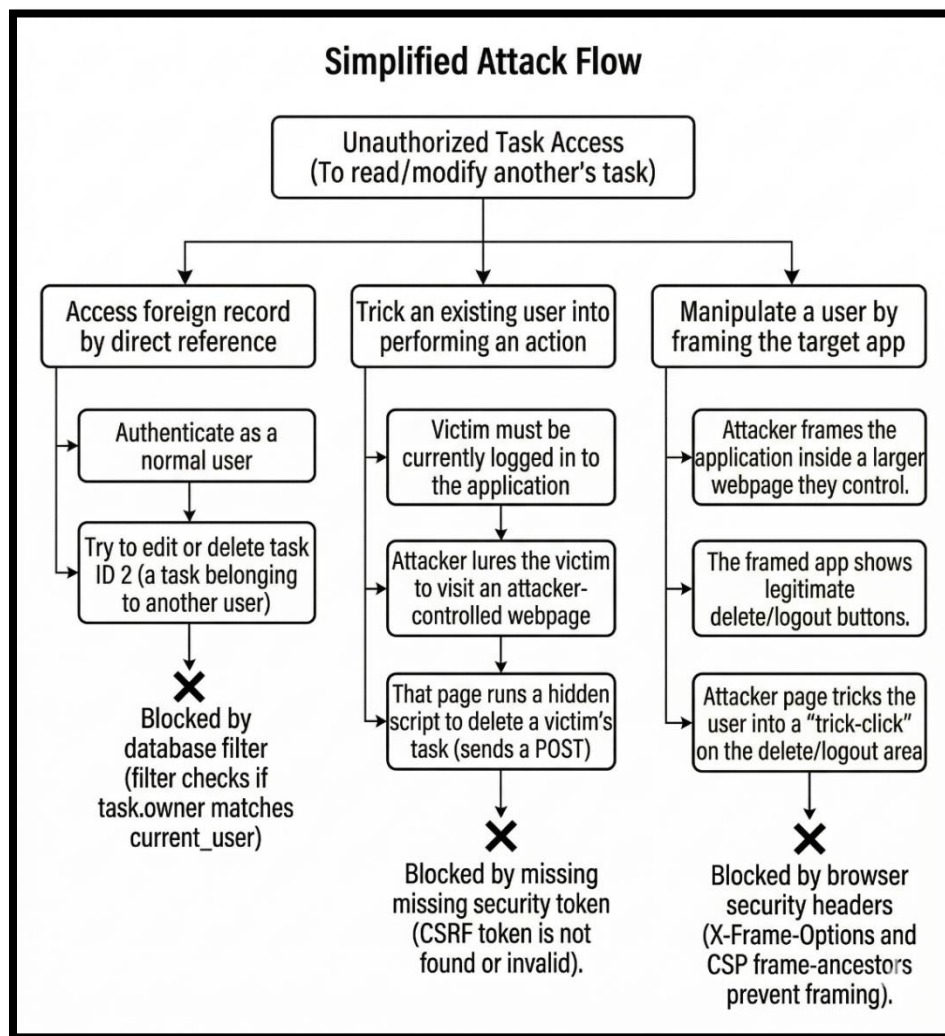


3.1 Trust Boundaries

Boundary	From	To	Nature
TB1 — Untrusted Input	Browser	Flask Application	All inputs must be validated and sanitized
TB2 — Trusted App/Data	Flask App	SQLite Database	Application-layer controls enforced before DB write
TB3 — Client Storage	Session Cookie	Browser	Client-controlled; protected by SameSite=Lax and HttpOnly

4. Attack Tree — Unauthorized Task Access

The following attack tree maps the exploitation paths an adversary could use to read or modify another user's task. All three paths are currently mitigated by implemented controls.



5. Implemented Mitigations

The following controls have been implemented and verified in the current codebase:

Control	Vulnerability Addressed	Implementation Detail	Status
Object-level ownership check	IDOR	<code>_get_user_task_or_404()</code> filters Task by <code>task_id</code> AND <code>current_user.id</code>	Implemented
CSRF protection	CSRF	<code>csrf.init_app(app)</code> + hidden <code>csrf_token</code> in all auth and task forms	Implemented
Logout hardening	CSRF on logout	<code>/logout</code> changed to POST method with CSRF token in navbar form	Implemented
Session cookie hardening	Session hijack / CSRF	<code>SESSION_COOKIE_SAMESITE = 'Lax'</code> set in <code>config.py</code>	Implemented
Anti-framing headers	Clickjacking	<code>app.after_request</code> sets X-Frame-Options: DENY and CSP frame-ancestors 'none'	Implemented

6. Residual Gaps and Recommended Actions

The following gaps remain open and should be addressed before production deployment:

Gap ID	Risk Area	Current State	Recommended Action	Priority
G-01	Information Disclosure	<code>run.py</code> starts Flask with <code>debug=True</code>	Set <code>DEBUG=False</code> via environment variable in all non-development contexts	High
G-02	Denial of Service / Spoofing	No rate limit or lockout on <code>/login</code>	Integrate Flask-Limiter to throttle login attempts; add exponential back-off or account lockout	High
G-03	Repudiation	No persistent audit log	Implement structured audit logging capturing <code>user_id</code> , IP address, timestamp, HTTP method, and route for all auth and task CRUD events	Medium

7. CVSS v3.1 Risk Ratings

The following CVSS v3.1 base scores were assigned to the three primary vulnerabilities. Scores reflect the exploitability and impact characteristics of each finding in the context of this application.

Vulnerability	CVSS v3.1 Vector	Base Score	Severity	Remediation Priority
CSRF — Unauthorized form submission	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	8.8	High	P1 — Addressed (CSRFProtect)
IDOR — Unauthorized task access	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N	6.5	Medium	P2 — Addressed (ownership filter)
Clickjacking	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	5.4	Medium	P3 — Addressed (XFO + CSP)

Score Rationale

- CSRF (8.8 High): No privileges required; any network attacker can craft the forged request. Integrity impact is high as any POST action can be triggered without the user's knowledge.
- IDOR (6.5 Medium): Requires a low-privilege authenticated account. Direct high integrity impact as any task record can be modified or deleted by a non-owner.
- Clickjacking (5.4 Medium): Requires user interaction and produces partial confidentiality and integrity impact via UI deception. Scope change (S:C) reflects browser context boundary crossing.

8. Summary and Conclusion

The Flask To-Do application implements a solid baseline of security controls across its three primary threat vectors. IDOR, CSRF, and Clickjacking are all mitigated through code-level controls verified in the current codebase.

The three residual gaps — debug mode enabled in production execution paths, absent login rate limiting, and no audit logging — should be addressed in the next sprint before the application is deployed to any non-development environment.

Area	Posture
IDOR protection	Mitigated — user-scoped object lookup on all task mutation routes
CSRF protection	Mitigated — global CSRFProtect + tokenized forms including logout
Clickjacking protection	Mitigated — X-Frame-Options: DENY + CSP frame-ancestors 'none'
Session security	Partially hardened — SameSite=Lax set; HttpOnly and Secure flags should be confirmed
Debug mode	Gap — debug=True in run.py must be removed for production
Login rate limiting	Gap — no throttling on /login; brute-force risk remains open
Audit logging	Gap — no persistent event log; repudiation risk remains open